

DEEP LEARNING

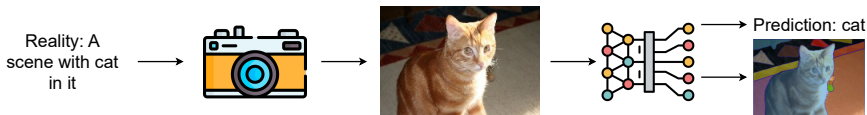
Deep Learning for Computer Vision: Part II

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Recap: What is Computer Vision?

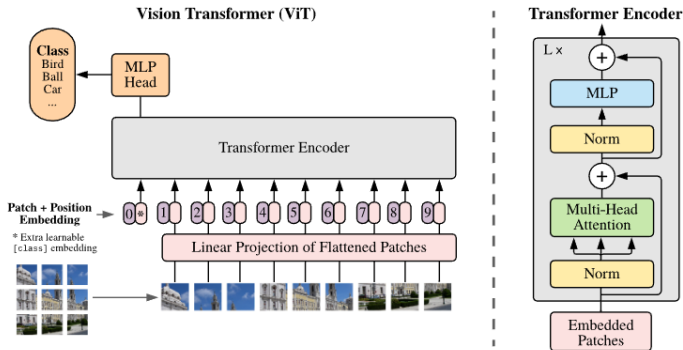
- **Goal:** Enable machines to see, understand, and act on visual data using neural networks.
- Core idea in computer vision:
 - ▶ Image in → prediction out (eg, one label per image or one label per pixel)
 - ▶ Learn features automatically using a deep neural network
 - ▶ Hierarchical representation: We go from pixels → edges → textures → object parts → objects



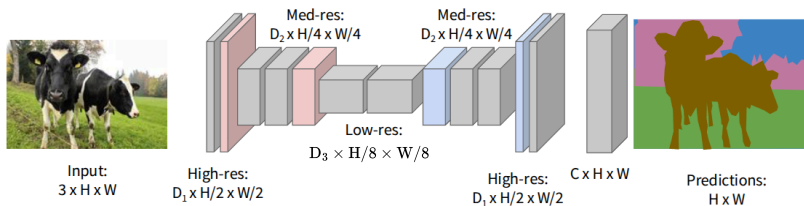
- Applications:
 - ▶ Autonomous driving
 - ▶ Medical imaging
 - ▶ Surveillance

Recap: Transformer for Image Classification

- We can use Transformer architecture, originally designed for NLP, for the task of image classification.
- The only modification needed is to convert the input image to smaller patches (eg, 16×16) and then convert them into embeddings (or vectors).



Recap: Fully Convolutional Network for Segmentation



- A fully convolutional network has only convolutional layer, and no MLP component. It has the following two parts:
 - ▶ a **downsampling** part (just as a regular CNN for image classification)
 - ▶ a **upsampling** part
- The downsampling part of the network produces lower resolution features: $D_3 \times H/8 \times W/8$.
- The upsampling part of the network "upsamples" the lower resolution features back to the resolution of the image (or the ground truth): $H \times W$.

Today

- CNNs are versatile and can be used for tasks beyond image classification, such as:
 - ▶ Object detection (**today**)
 - ▶ Segmentation
 - ▶ Depth estimation
 - ▶ ... many more
- Transformers although originally developed for the task of NLP, can be used for Computer Vision tasks:
 - ▶ Image classification
 - ▶ Segmentation
 - ▶ Object detection
 - ▶ ... many more



Segmentation



Object detection



Depth estimation

Table of Contents

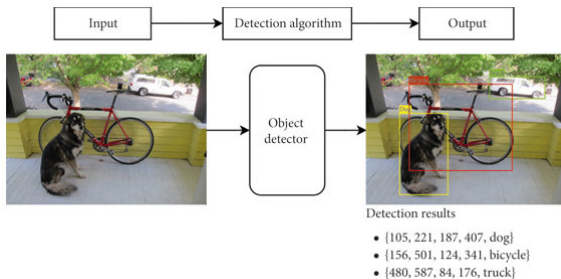
1. Two-stage Object Detector

- 1.1 R-CNN
- 1.2 Fast R-CNN
- 1.3 Faster R-CNN

2. Single-Stage Object Detector

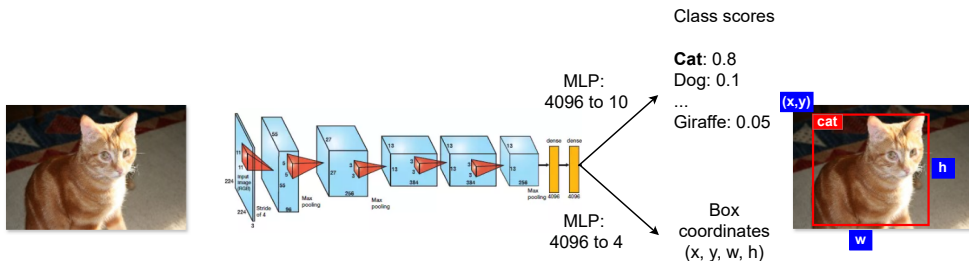
- 2.1 YOLO

What is Object Detection?



- **Goal:** Locate **what** objects are present and **where** they are in the image.
- It combines classification (what) and localization (where).
- Input: an image; Output: bounding boxes + class labels for each detected object.
- Types of object detectors:
 - ▶ Two-stage detectors: R-CNN, Faster R-CNN
 - ▶ One-stage detectors: YOLO, SSD
 - ▶ Transformer-based detectors: DETR

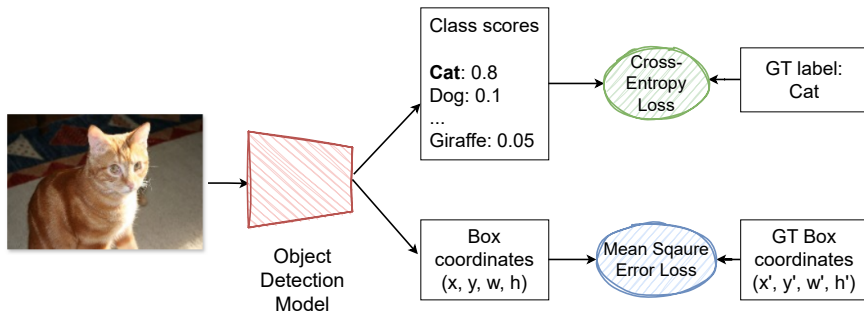
Object Detection: A Simple Approach



- We can use a standard CNN (eg, Alexnet) and re-purpose it for object detection:
 - ▶ **Classification head:** It predicts a probability distribution over the classes (assuming there are 10 classes in the dataset) → answers the question *what*
 - ▶ **Bounding box head:** It predicts the box coordinates, ie, the four values: x, y, w, h , which are the coordinates of the leftmost corner, the height and width of the box, respectively → answers the question *where*

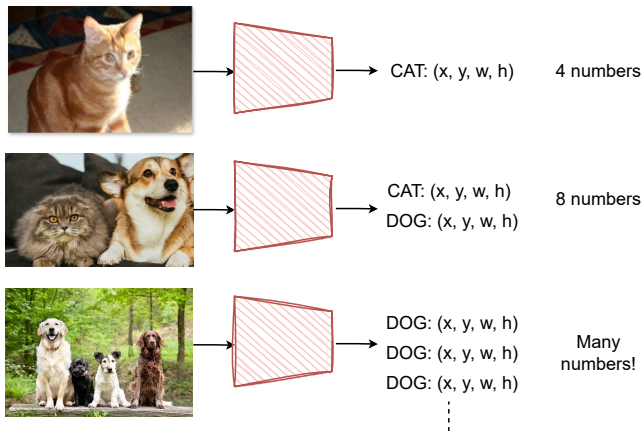
Object Detection: A Simple Approach

- We can train the object detection model using a combination of two losses:
 - ▶ Cross entropy loss to predict the correct class label → **classification task**
 - ▶ Mean square error loss to predict the correct box coordinates → **regression task**



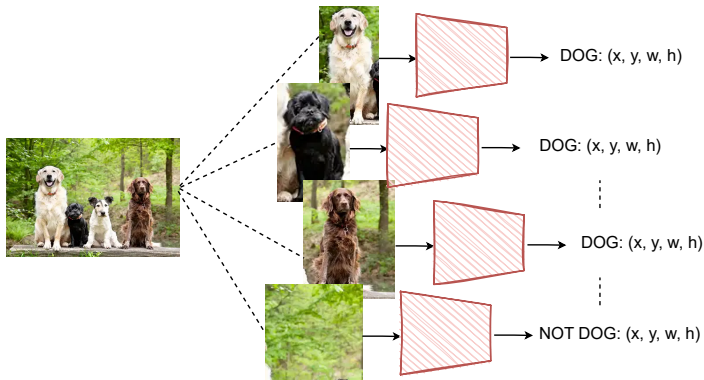
- **Question:** What is the main drawback of this approach?

Object Detection: The Need to Handle Multiple Objects



- Depending on the content of an image, we need to output different number of outputs! Current architecture can not handle different number of outputs.

Object Detection: A Sliding Window Approach

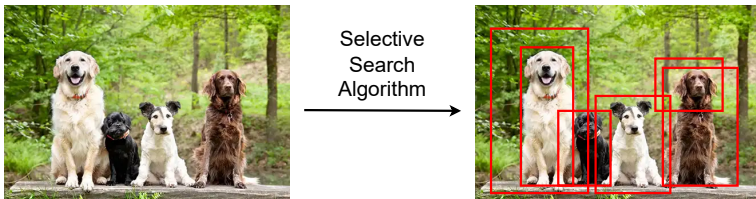


- We can slide small windows of different scales and aspect ratios across different locations of the image in order to detect multiple objects.
- Downside of such an approach is that we need to apply CNN at a huge number of scales, aspect ratios and locations, making it computationally wasteful!

R-CNN: Regions with CNN features

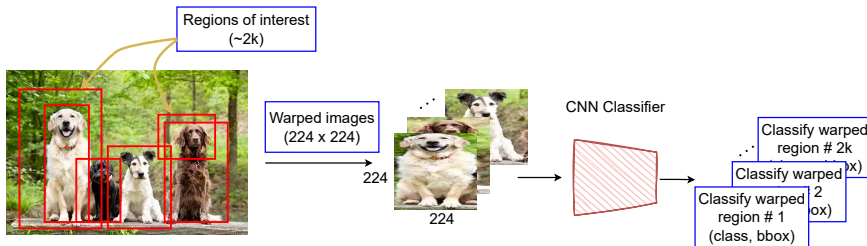
R-CNN

- Instead trying all possible image crops, we first find potential objects (or blobby image regions) in the input image¹.
- For that we apply the [selective search algorithm](#) that outputs image regions that are likely to contain an object.
- Selective search gives around 2000 region proposals.



¹R-CNN paper

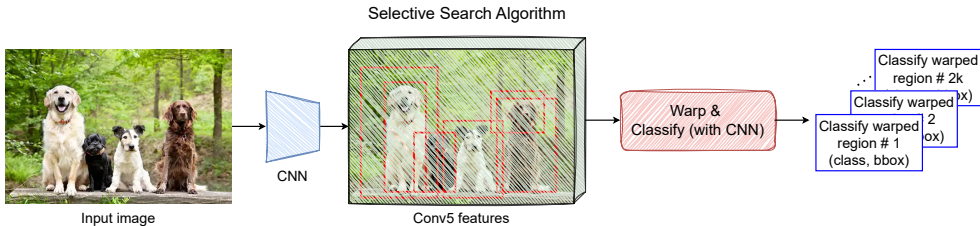
R-CNN



- We warp (ie, resize and reshape) each region of interest as a regular sized image (eg, 224×224) and provide as an independent input to the network.
- For each warped image, the CNN outputs the class label and bbox coordinates.
- Although it is a better approach than brute forcing predictions at all locations, it is still slow as it needs to process $\sim 2k$ images independently (or in several mini-batches). Can we do better?

Fast R-CNN

Fast R-CNN



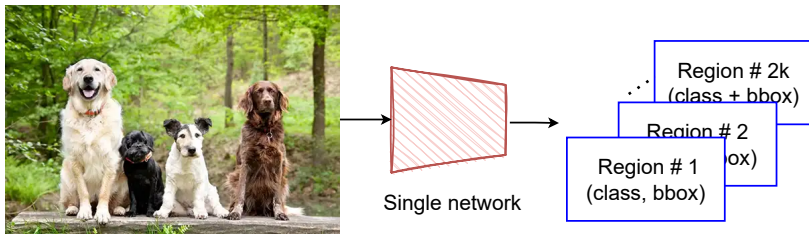
- To speed up the inference, Fast R-CNN² was proposed that runs that whole image through a Convnet only once, and applies selective search algorithm on top of the conv 5 features.
- The rest of the pipeline remains unchanged from R-CNN, ie, warping the features and then applying a CNN on the warped features to predict class labels + bbox coordinates.

²Fast R-CNN paper

Faster R-CNN

Faster R-CNN

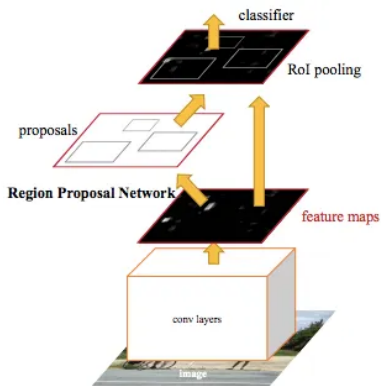
- Finding **Regions of Interest** (Rols) through selective search algorithm, either at the input or on top of the conv5 features, makes the pipeline cumbersome.
- We want to make learning **end-to-end**, where we want to obtain Rols and classification in one single step.
- Faster R-CNN paper³ proposed to replace the selective search algorithm with **Region Proposal Network** (RPN).



³Faster R-CNN paper

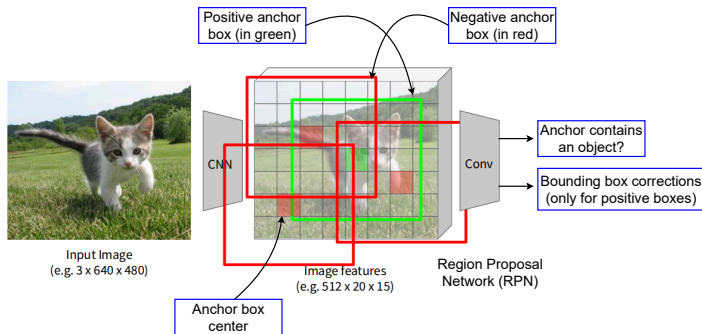
Region Proposal Network

- Region Proposal Network (RPN) is a small network that generates the so called "proposals" or regions of interest, the job that was earlier done by the selective search algorithm.
- RPN simply slides over the convolutional features obtained from the backbone, and proposes 2k regions of interest.
- **Question:** How can we train the RPN?



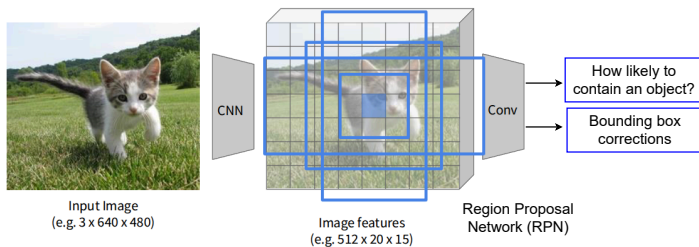
Region Proposal Network: Training

- We train in the following manner:
 - ▶ We uniformly sample K **anchor boxes** at different locations with different scale and aspect ratios.
 - ▶ For each anchor box we ask the network if there is an object or not, ie, binary classification.
 - ▶ For positive anchor boxes (the ones that actually contain the object), we also ask the network to predict the bounding box corrections.

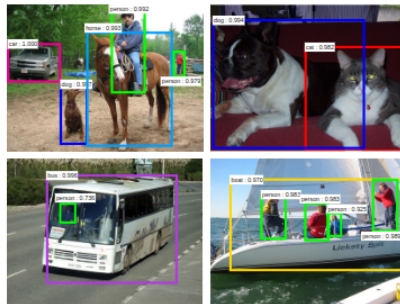
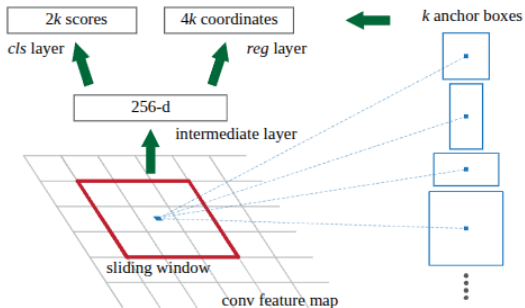


Region Proposal Network: Inference

- Inference in RPN works as follows:
 - ▶ For K anchor boxes, predict the "objectness" score or how likely the anchor contains an object + bounding box coordinates.
 - ▶ Sort the predictions based on the highest "objectness" scores, and take top ~ 300 as our proposals.
- After getting the regions of interest, we follow the exact same pipeline as before: warping (or region of interest pooling) and then classifying for each RoI the class and the bounding box coordinates.

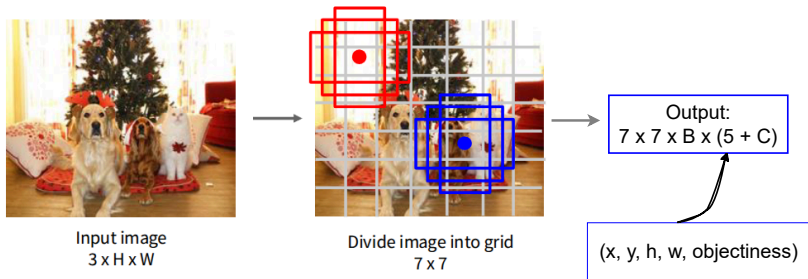


Region Proposal Network: Inference



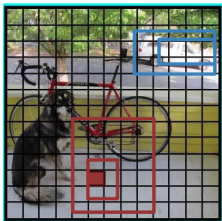
Single-Stage Object Detector

Single-Stage Object Detector



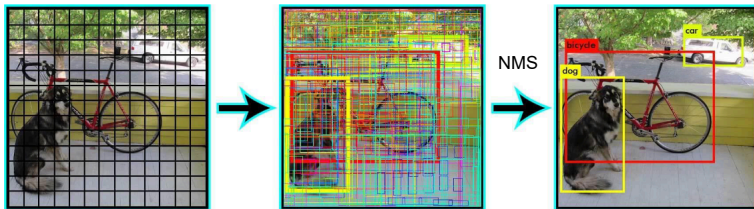
- The working principle of Single-Stage Detectors: Directly from image pixels $\rightarrow$ to bounding box coordinates + class probabilities.
 - ▶ Divide an image into grid of 7×7 . Each grid cell will have B anchor boxes of different scale and aspect ratio. In this example, $B = 3$.
 - ▶ For each grid cell predict:
 - Bounding box coordinates (the five values): x, y, h, w and objectness score
 - Scores for each of C classes (including a background class)

YOLO: You Only Look Once



- For each box, the network outputs:
 - ▶ $P(\text{object})$: probability that the box contains an object, ie, the objectness score
 - ▶ Bounding box coordinates (x, y, h, w)
 - ▶ $P(\text{class})$: probability of belonging to a class
- Several variants of YOLO so far:
 - ▶ $\text{YOLOv1} = 7 \times 7 \times \underbrace{2}_{\text{boxes}} \times (5 + \underbrace{20}_{\text{classes}}) = 2450$
 - ▶ YOLOv3 has multiple detection scales, ie, 13×13 , 26×26 , 52×52 . We make $S \times S \times \underbrace{3}_{\text{boxes}} \times (5 + \underbrace{80}_{\text{classes}})$ boxes per scale.

Non-Maximum Suppression



- We generally get a lot of predictions since there are several proposal boxes.
- To remove the clutter, we apply non-maximum suppression (NMS).
- Intuitively, NMS works as follows:
 - ▶ Sort the predicted bounding boxes based on confidence (or objectness) score.
 - ▶ Select the top-scoring box, call it A .
 - ▶ Remove all other boxes that have a high overlap with A .
 - ▶ Repeat the above steps with the next highest remaining box.

Summary

- The goal of object detection is to locate what objects are in the image and where they are, ie, bounding boxes + class labels.
- Two kinds of object detection methods:
 - ▶ Two-stage object detectors: generate region proposals and then refine these proposals (eg, Faster R-CNN). Typically higher in accuracy but slower.
 - ▶ Single-stage detectors: detect objects directly in one pass, ie, no proposal stage (eg, YOLO). Typically much faster but less accurate.
- Next time, we will see how transformer-based object detection and segmentation methods work.

References and Links

- For additional reading materials refer to the following links
 - ▶ [“Dive into Deep Learning”](#)⁴, Chapters: 14 (14.3-14.8).
 - ▶ [“Presentation from Ross Girshick”](#)⁵ (nobody understands object detection better than him)
- Some slides have been adapted from E. Adeli.

⁴<https://d2l.ai/>

⁵<https://dl.dropboxusercontent.com/s/vlyrkgd8nz8gy5l/fast-rcnn.pdf?dl=0>